

# Dynamic Programming: DP on Grids

A Problem-Solving Companion to Striver's A2Z DSA Sheet

Maruri Saikiran

June 29, 2026



# Contents

|          |                                                         |          |
|----------|---------------------------------------------------------|----------|
| <b>1</b> | <b>Dynamic Programming: 2D/3D DP and DP on Grids</b>    | <b>1</b> |
| 1.1      | From a Line to a Grid . . . . .                         | 1        |
| 1.2      | Unique Paths . . . . .                                  | 2        |
| 1.3      | Unique Paths II (With Obstacles) . . . . .              | 5        |
| 1.4      | Minimum Path Sum . . . . .                              | 8        |
| 1.5      | Triangle . . . . .                                      | 11       |
| 1.6      | Ninja's Training . . . . .                              | 14       |
| 1.7      | Ninja and his Friends (3D DP / Cherry Pickup) . . . . . | 18       |



# Chapter 1

## Dynamic Programming: 2D/3D DP and DP on Grids

*A 1D DP table has one knob: which earlier position(s) a state depends on. A grid DP table has two – row and column – and the only genuinely new idea in this chapter is that a cell's recurrence now reads from neighbors in two directions instead of earlier points on one line. Everything else, all four stages, transfers unchanged.*

### 1.1 From a Line to a Grid

Every problem in the DP 1D chapter (Fibonacci through House Robber II) indexed its DP state by a single integer (a position along an array). This chapter's problems index state by **two** integers (a row and a column), because the underlying problem is itself naturally laid out on a grid. The four-stage methodology from the DP 1D chapter's introduction – recursion, memoization, tabulation, space optimization – applies identically; only the *shape* of the memo array and the tabulation loop changes, from a 1D array scanned in one direction to a 2D array filled row by row (or column by column).

#### How to Recognise a Grid DP Problem and Its Recurrence

- **“Number of paths / minimum-cost path from top-left to bottom-right, moving only right or down”** – the recurrence at cell  $(i, j)$  combines the answers at  $(i-1, j)$  (arrived from above) and  $(i, j-1)$  (arrived from the left), summed for counting paths, or minimized/maximized for cost.
- **“Minimum/maximum path sum from the top of a triangle (or any row) to the bottom”** – the recurrence at  $(i, j)$  combines the two cells reachable *from* it in the next row (or the two reachable *into* it from the previous row, depending on which direction the recursion scans), rather than from directly above/left – the exact indices depend on whether you formulate top-down or bottom-up first.
- **“Choose one of several options each day/step, but the same option cannot be chosen on two consecutive days/steps”** – a state with *two* dimensions where one is not spatial at all: the position (day/step) and *which option was just used*, so the recurrence can exclude the previous day's choice when picking the best current option.
- **“Two agents/paths moving simultaneously through the same grid”** – a 3D DP state (one shared row/step index, plus one column index per agent), since both agents' positions must be tracked together to correctly avoid double-counting cells they both visit.

This chapter moves through exactly these four variations in order, starting with the cleanest two-direction grid recurrence (Unique Paths) and ending with the full 3D, two-agent recurrence (Ninja and his Friends), which is this chapter's capstone, exactly as Largest BST in a Binary Tree and the LFU Cache closed their respective chapters.

## 1.2 Unique Paths

**Problem Statement.** A robot starts at the top-left corner of an  $m \times n$  grid and wants to reach the bottom-right corner, moving only **right** or **down** at each step. Count the number of unique paths.

### Pattern Recognition

“Number of paths through a grid, moving only right or down” is the cleanest possible grid-DP recurrence: a path arrives at cell  $(i, j)$  from exactly one of two neighbors – directly above,  $(i - 1, j)$ , or directly to the left,  $(i, j - 1)$  – and since every path to  $(i, j)$  is either one of these two cases, the count of paths to  $(i, j)$  is their **sum**, exactly mirroring the DP 1D chapter’s Climbing Stairs “sum over a fixed set of predecessors,” just with the predecessors now arranged on two grid axes instead of one line.

### Deriving the recurrence

Let  $ways(i, j)$  be the number of unique paths from  $(0, 0)$  to  $(i, j)$ . Since the only moves are right and down, the last move into  $(i, j)$  came from  $(i - 1, j)$  (a down-move) or  $(i, j - 1)$  (a right-move):

$$ways(i, j) = ways(i - 1, j) + ways(i, j - 1).$$

Base case:  $ways(0, 0) = 1$  (the trivial path of staying put); any  $ways(i, j)$  with  $i < 0$  or  $j < 0$  is treated as 0 (non-existent, out-of-grid predecessors contribute nothing).

### Approach 1: Recursion

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int uniquePathsRecursive(int i, int j) {
5     if (i == 0 && j == 0) return 1;
6     if (i < 0 || j < 0) return 0;
7
8     int fromAbove = uniquePathsRecursive(i - 1, j);
9     int fromLeft = uniquePathsRecursive(i, j - 1);
10    return fromAbove + fromLeft;
11 }
12
13 int main() {
14     int m = 3, n = 7;
15     cout << uniquePathsRecursive(m - 1, n - 1) << endl; // 28
16     return 0;
17 }
```

### Approach 1 Complexity

**Time.**  $O(2^{m+n})$  – each call branches into two, with recursion depth up to  $m + n$ . **Space.**  $O(m + n)$  for the recursion stack.

### Approach 2: Memoization

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int uniquePathsMemo(int i, int j, vector<vector<int>>& memo) {
5     if (i == 0 && j == 0) return 1;
6     if (i < 0 || j < 0) return 0;
7     if (memo[i][j] != -1) return memo[i][j];
8
9     int fromAbove = uniquePathsMemo(i - 1, j, memo);
10    int fromLeft = uniquePathsMemo(i, j - 1, memo);
11    return memo[i][j] = fromAbove + fromLeft;
12 }
13
14 int main() {
15     int m = 3, n = 7;
16     vector<vector<int>> memo(m, vector<int>(n, -1));
17     cout << uniquePathsMemo(m - 1, n - 1, memo) << endl; // 28
18     return 0;
19 }

```

### Approach 2 Complexity

**Time.**  $O(m \cdot n)$  – one  $O(1)$  computation per distinct  $(i, j)$  pair. **Space.**  $O(m \cdot n)$  memo +  $O(m + n)$  recursion stack.

### Approach 3: Tabulation

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int uniquePathsTabulation(int m, int n) {
5     vector<vector<int>> dp(m, vector<int>(n, 0));
6
7     for (int i = 0; i < m; i++) {
8         for (int j = 0; j < n; j++) {
9             if (i == 0 && j == 0) {
10                 dp[i][j] = 1;
11                 continue;
12             }
13             int fromAbove = (i > 0) ? dp[i-1][j] : 0;
14             int fromLeft = (j > 0) ? dp[i][j-1] : 0;
15             dp[i][j] = fromAbove + fromLeft;
16         }
17     }
18     return dp[m - 1][n - 1];
19 }
20
21 int main() {
22     cout << uniquePathsTabulation(3, 7) << endl; // 28
23     return 0;
24 }

```

### Dry run of the tabulation table

**Dry Run.** Take  $m = 3, n = 3$ .

|         | $j = 0$ | $j = 1$ | $j = 2$ |
|---------|---------|---------|---------|
| $i = 0$ | 1       | 1       | 1       |
| $i = 1$ | 1       | 2       | 3       |
| $i = 2$ | 1       | 3       | 6       |

The first row and first column are all 1s (only one way to reach any cell along the top edge or left edge: a straight line of moves). Every other cell is the sum of its top and left neighbors, e.g.  $dp[2][2] = dp[1][2] + dp[2][1] = 3 + 3 = 6$ . The answer for a  $3 \times 3$  grid is 6.

### Approach 3 Complexity

**Time.**  $O(m \cdot n)$ . **Space.**  $O(m \cdot n)$  table, no recursion stack.

## Approach 4: Space Optimization

### Only the Previous Row Is Ever Needed

Computing row  $i$  only reads from row  $i - 1$  (directly above) and from *within* row  $i$  itself (directly left, already computed earlier in the same row's left-to-right scan). No row before  $i - 1$  is ever needed again once row  $i$  is being filled, so a single 1D array, reused and updated in place across rows, suffices.

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int uniquePathsSpaceOptimized(int m, int n) {
5     vector<int> prevRow(n, 0);
6
7     for (int i = 0; i < m; i++) {
8         vector<int> currentRow(n, 0);
9         for (int j = 0; j < n; j++) {
10             if (i == 0 && j == 0) {
11                 currentRow[j] = 1;
12                 continue;
13             }
14             int fromAbove = prevRow[j];
15             int fromLeft = (j > 0) ? currentRow[j-1] : 0;
16             currentRow[j] = fromAbove + fromLeft;
17         }
18         prevRow = currentRow;
19     }
20     return prevRow[n - 1];
21 }
22
23 int main() {
24     cout << uniquePathsSpaceOptimized(3, 7) << endl; // 28
25     return 0;
26 }
```

### Approach 4 Complexity

**Time.**  $O(m \cdot n)$ . **Space.**  $O(n)$  for the single rolling row, instead of  $O(m \cdot n)$  for the full table.

**Key Takeaway**

Grid DP's space optimization mirrors 1D DP's exactly, just one dimension up: instead of two rolling *scalars* (Fibonacci, House Robber), we keep two rolling *rows* (or, with more care, overwrite a single row in place, since the left-neighbor read within a row happens before that position is overwritten in a left-to-right scan). Whenever a grid recurrence only reads from the immediately preceding row and the current row, this reduction from  $O(mn)$  to  $O(n)$  space is always available.

## 1.3 Unique Paths II (With Obstacles)

**Problem Statement.** Same as Section 1.2, except some grid cells are marked as **obstacles** (value 1; open cells are 0), and the robot cannot step onto an obstacle cell at all. Count the number of unique paths from top-left to bottom-right.

**Pattern Recognition**

“Same grid-path-counting problem, with some cells forbidden” needs exactly one change to Section 1.2's recurrence: an obstacle cell contributes **zero** paths, regardless of what its predecessors computed, because no path is allowed to pass through it at all.

### Deriving the recurrence

$$ways(i, j) = \begin{cases} 0 & \text{if } (i, j) \text{ is an obstacle,} \\ ways(i-1, j) + ways(i, j-1) & \text{otherwise.} \end{cases}$$

The base case also needs the obstacle check:  $ways(0, 0) = 1$  *unless*  $(0, 0)$  itself is an obstacle, in which case no path can even begin, and the answer is 0 overall.

### Approach 1: Recursion

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int uniquePaths2Recursive(int i, int j, vector<vector<int>>& grid) {
5     if (i < 0 || j < 0 || grid[i][j] == 1) return 0;
6     if (i == 0 && j == 0) return 1;
7
8     int fromAbove = uniquePaths2Recursive(i - 1, j, grid);
9     int fromLeft = uniquePaths2Recursive(i, j - 1, grid);
10    return fromAbove + fromLeft;
11 }
12
13 int main() {
14     vector<vector<int>> grid = {{0,0,0},{0,1,0},{0,0,0}};
15     int m = grid.size(), n = grid[0].size();
16     cout << uniquePaths2Recursive(m - 1, n - 1, grid) << endl; // 2
17     return 0;
18 }
```

**Approach 1 Complexity**

**Time.**  $O(2^{m+n})$ . **Space.**  $O(m + n)$  recursion stack.

## Approach 2: Memoization

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int uniquePaths2Memo(int i, int j, vector<vector<int>>& grid,
5                     vector<vector<int>>& memo) {
6     if (i < 0 || j < 0 || grid[i][j] == 1) return 0;
7     if (i == 0 && j == 0) return 1;
8     if (memo[i][j] != -1) return memo[i][j];
9
10    int fromAbove = uniquePaths2Memo(i - 1, j, grid, memo);
11    int fromLeft = uniquePaths2Memo(i, j - 1, grid, memo);
12    return memo[i][j] = fromAbove + fromLeft;
13 }
14
15 int main() {
16     vector<vector<int>> grid = {{0,0,0},{0,1,0},{0,0,0}};
17     int m = grid.size(), n = grid[0].size();
18     vector<vector<int>> memo(m, vector<int>(n, -1));
19     cout << uniquePaths2Memo(m - 1, n - 1, grid, memo) << endl; // 2
20     return 0;
21 }

```

### Approach 2 Complexity

**Time.**  $O(m \cdot n)$ . **Space.**  $O(m \cdot n)$  memo +  $O(m + n)$  recursion stack.

## Approach 3: Tabulation

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int uniquePaths2Tabulation(vector<vector<int>>& grid) {
5     int m = grid.size(), n = grid[0].size();
6     vector<vector<int>> dp(m, vector<int>(n, 0));
7
8     for (int i = 0; i < m; i++) {
9         for (int j = 0; j < n; j++) {
10             if (grid[i][j] == 1) {
11                 dp[i][j] = 0;
12                 continue;
13             }
14             if (i == 0 && j == 0) {
15                 dp[i][j] = 1;
16                 continue;
17             }
18             int fromAbove = (i > 0) ? dp[i-1][j] : 0;
19             int fromLeft = (j > 0) ? dp[i][j-1] : 0;
20             dp[i][j] = fromAbove + fromLeft;
21         }
22     }
23     return dp[m - 1][n - 1];
24 }
25
26 int main() {

```

```

27     vector<vector<int>> grid = {{0,0,0},{0,1,0},{0,0,0}};
28     cout << uniquePaths2Tabulation(grid) << endl; // 2
29     return 0;
30 }

```

### Dry run of the tabulation table

**Dry Run.** Take  $grid = \begin{bmatrix} 0 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 0 \end{bmatrix}$ .

|         | $j = 0$ | $j = 1$      | $j = 2$ |
|---------|---------|--------------|---------|
| $i = 0$ | 1       | 1            | 1       |
| $i = 1$ | 1       | 0 (obstacle) | 1       |
| $i = 2$ | 1       | 1            | 2       |

At (1, 1), the obstacle forces  $dp = 0$  regardless of its neighbors. At (2, 2):  $dp[1][2] + dp[2][1] = 1 + 1 = 2$ . The answer is 2: the two paths route either entirely along the top then down the right edge, or entirely down the left edge then along the bottom.

### Approach 3 Complexity

**Time.**  $O(m \cdot n)$ . **Space.**  $O(m \cdot n)$  table, no recursion stack.

### Approach 4: Space Optimization

```

1  #include <bits/stdc++.h>
2  using namespace std;
3
4  int uniquePaths2SpaceOptimized(vector<vector<int>>& grid) {
5      int m = grid.size(), n = grid[0].size();
6      vector<int> prevRow(n, 0);
7
8      for (int i = 0; i < m; i++) {
9          vector<int> currentRow(n, 0);
10         for (int j = 0; j < n; j++) {
11             if (grid[i][j] == 1) {
12                 currentRow[j] = 0;
13                 continue;
14             }
15             if (i == 0 && j == 0) {
16                 currentRow[j] = 1;
17                 continue;
18             }
19             int fromAbove = prevRow[j];
20             int fromLeft = (j > 0) ? currentRow[j-1] : 0;
21             currentRow[j] = fromAbove + fromLeft;
22         }
23         prevRow = currentRow;
24     }
25     return prevRow[n - 1];
26 }
27
28 int main() {
29     vector<vector<int>> grid = {{0,0,0},{0,1,0},{0,0,0}};

```

```

30     cout << uniquePaths2SpaceOptimized(grid) << endl; // 2
31     return 0;
32 }

```

### Approach 4 Complexity

**Time.**  $O(m \cdot n)$ . **Space.**  $O(n)$  for the rolling row.

### Key Takeaway

Adding a “forbidden state” constraint to an already-derived DP recurrence is almost always a single extra case at the very top of the recurrence (here, “obstacle  $\Rightarrow$  zero,” checked before anything else), with every other part of the four-stage derivation unchanged. This is the same kind of small, composable extension already seen when Combination Sum III added a length constraint to Combination Sum II’s base case.

## 1.4 Minimum Path Sum

**Problem Statement.** Given an  $m \times n$  grid of non-negative numbers, find a path from the top-left to the bottom-right corner, moving only right or down, that **minimizes** the sum of the numbers along the path.

### Pattern Recognition

This is Unique Paths (Section 1.2) with sum replaced by minimum – exactly the same transformation that turned Climbing Stairs into Frog Jump back in the DP 1D chapter. The recurrence’s *structure* (combine the cell above and the cell to the left) is identical; only the combination operator and the addition of the current cell’s own value change.

### Deriving the recurrence

Let  $\text{minSum}(i, j)$  be the minimum path sum to reach  $(i, j)$ . The path’s last step into  $(i, j)$  came from  $(i - 1, j)$  or  $(i, j - 1)$ , and we must add the current cell’s own value to whichever predecessor gives the smaller total:

$$\text{minSum}(i, j) = \text{grid}[i][j] + \min(\text{minSum}(i - 1, j), \text{minSum}(i, j - 1)).$$

Base case:  $\text{minSum}(0, 0) = \text{grid}[0][0]$ . For cells along the top row or left column, only one predecessor exists (the other is out of bounds and treated as  $+\infty$ , never the chosen minimum).

### Approach 1: Recursion

```

1  #include <bits/stdc++.h>
2  using namespace std;
3
4  int minPathSumRecursive(int i, int j, vector<vector<int>>& grid) {
5      if (i == 0 && j == 0) return grid[0][0];
6      if (i < 0 || j < 0) return INT_MAX;
7
8      int fromAbove = minPathSumRecursive(i - 1, j, grid);
9      int fromLeft  = minPathSumRecursive(i, j - 1, grid);
10     return grid[i][j] + min(fromAbove, fromLeft);
11 }
12

```

```

13 int main() {
14     vector<vector<int>> grid = {{1,3,1},{1,5,1},{4,2,1}};
15     int m = grid.size(), n = grid[0].size();
16     cout << minPathSumRecursive(m - 1, n - 1, grid) << endl; // 7
17     return 0;
18 }

```

### Approach 1 Complexity

**Time.**  $O(2^{m+n})$ . **Space.**  $O(m + n)$  recursion stack.

### Approach 2: Memoization

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int minPathSumMemo(int i, int j, vector<vector<int>>& grid,
5                     vector<vector<int>>& memo) {
6     if (i == 0 && j == 0) return grid[0][0];
7     if (i < 0 || j < 0) return INT_MAX;
8     if (memo[i][j] != -1) return memo[i][j];
9
10    int fromAbove = minPathSumMemo(i - 1, j, grid, memo);
11    int fromLeft = minPathSumMemo(i, j - 1, grid, memo);
12    return memo[i][j] = grid[i][j] + min(fromAbove, fromLeft);
13 }
14
15 int main() {
16     vector<vector<int>> grid = {{1,3,1},{1,5,1},{4,2,1}};
17     int m = grid.size(), n = grid[0].size();
18     vector<vector<int>> memo(m, vector<int>(n, -1));
19     cout << minPathSumMemo(m - 1, n - 1, grid, memo) << endl; // 7
20     return 0;
21 }

```

### Approach 2 Complexity

**Time.**  $O(m \cdot n)$ . **Space.**  $O(m \cdot n)$  memo +  $O(m + n)$  recursion stack.

### Approach 3: Tabulation

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int minPathSumTabulation(vector<vector<int>>& grid) {
5     int m = grid.size(), n = grid[0].size();
6     vector<vector<int>> dp(m, vector<int>(n, 0));
7
8     for (int i = 0; i < m; i++) {
9         for (int j = 0; j < n; j++) {
10             if (i == 0 && j == 0) {
11                 dp[i][j] = grid[0][0];
12                 continue;
13             }
14             int fromAbove = (i > 0) ? dp[i-1][j] : INT_MAX;

```

```

15         int fromLeft = (j > 0) ? dp[i][j-1] : INT_MAX;
16         dp[i][j] = grid[i][j] + min(fromAbove, fromLeft);
17     }
18 }
19 return dp[m - 1][n - 1];
20 }
21
22 int main() {
23     vector<vector<int>> grid = {{1,3,1},{1,5,1},{4,2,1}};
24     cout << minPathSumTabulation(grid) << endl; // 7
25     return 0;
26 }

```

### Dry run of the tabulation table

**Dry Run.** Take  $grid = \begin{bmatrix} 1 & 3 & 1 \\ 1 & 5 & 1 \\ 4 & 2 & 1 \end{bmatrix}$ .

|         | $j = 0$ | $j = 1$ | $j = 2$ |
|---------|---------|---------|---------|
| $i = 0$ | 1       | 4       | 5       |
| $i = 1$ | 2       | 7       | 6       |
| $i = 2$ | 6       | 8       | 7       |

E.g.  $dp[1][2] = grid[1][2] + \min(dp[0][2], dp[1][1]) = 1 + \min(5, 7) = 1 + 5 = 6$ . The answer is  $dp[2][2] = 7$ , achieved by the path  $1 \rightarrow 3 \rightarrow 1 \rightarrow 1 \rightarrow 1$  (right, right, down, down), summing to  $1 + 3 + 1 + 1 + 1 = 7$ .

### Approach 3 Complexity

**Time.**  $O(m \cdot n)$ . **Space.**  $O(m \cdot n)$  table, no recursion stack.

### Approach 4: Space Optimization

```

1  #include <bits/stdc++.h>
2  using namespace std;
3
4  int minPathSumSpaceOptimized(vector<vector<int>>& grid) {
5      int m = grid.size(), n = grid[0].size();
6      vector<int> prevRow(n, 0);
7
8      for (int i = 0; i < m; i++) {
9          vector<int> currentRow(n, 0);
10         for (int j = 0; j < n; j++) {
11             if (i == 0 && j == 0) {
12                 currentRow[j] = grid[0][0];
13                 continue;
14             }
15             int fromAbove = prevRow[j];
16             int fromLeft = (j > 0) ? currentRow[j-1] : INT_MAX;
17             currentRow[j] = grid[i][j] + min(fromAbove, fromLeft);
18         }
19         prevRow = currentRow;
20     }
21     return prevRow[n - 1];

```

```

22 }
23
24 int main() {
25     vector<vector<int>> grid = {{1,3,1},{1,5,1},{4,2,1}};
26     cout << minPathSumSpaceOptimized(grid) << endl; // 7
27     return 0;
28 }

```

### Approach 4 Complexity

**Time.**  $O(m \cdot n)$ . **Space.**  $O(n)$  for the rolling row.

### Key Takeaway

Once a grid-path-counting recurrence is in hand, converting it into a grid-path-*cost* recurrence is purely mechanical: replace the sum of predecessors with a minimum (or maximum) of predecessors, plus the current cell's own contribution. Every later stage – memoization, tabulation, space optimization – carries over with zero additional changes, exactly as it did for the 1D Fibonacci-to-Frog-Jump transformation earlier in this book.

## 1.5 Triangle

**Problem Statement.** Given a triangular array of numbers (row  $i$  has  $i + 1$  entries, 0-indexed), find the minimum path sum from the top to any entry of the bottom row, where moving from  $(i, j)$  to the next row may go to  $(i + 1, j)$  or  $(i + 1, j + 1)$ .

### Pattern Recognition

“Path through a triangular grid, moving to one of two cells in the next row” is structurally the same combine-two-predecessors recurrence as Minimum Path Sum (Section 1.4), but with a twist worth noticing carefully: each cell's choices point **forward and downward** (to the next row), not backward (from the previous row), which means the most natural recursion starts at the **top** and recurses toward the bottom, and – correspondingly – the tabulation table is most naturally filled starting from the **bottom** row upward, the opposite direction from every grid DP problem so far in this chapter.

### Deriving the recurrence

Let  $minSum(i, j)$  be the minimum path sum starting from cell  $(i, j)$  down to *any* cell in the last row. From  $(i, j)$ , the path must continue to  $(i + 1, j)$  or  $(i + 1, j + 1)$ , and we want the cheaper continuation:

$$minSum(i, j) = triangle[i][j] + \min(minSum(i + 1, j), minSum(i + 1, j + 1)).$$

Base case: at the last row ( $i = n - 1$ ), there is nowhere further to go, so  $minSum(n - 1, j) = triangle[n - 1][j]$  directly. The overall answer is  $minSum(0, 0)$ .

### Approach 1: Recursion

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int triangleRecursive(int i, int j, vector<vector<int>>& triangle) {

```

```

5     int n = triangle.size();
6     if (i == n - 1) return triangle[i][j];
7
8     int down      = triangleRecursive(i + 1, j, triangle);
9     int downRight = triangleRecursive(i + 1, j + 1, triangle);
10    return triangle[i][j] + min(down, downRight);
11 }
12
13 int main() {
14     vector<vector<int>> triangle = {{2},{3,4},{6,5,7},{4,1,8,3}};
15     cout << triangleRecursive(0, 0, triangle) << endl; // 11
16     return 0;
17 }

```

### Approach 1 Complexity

**Time.**  $O(2^n)$  – two branches per call, depth  $n$ . **Space.**  $O(n)$  recursion stack.

### Approach 2: Memoization

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int triangleMemo(int i, int j, vector<vector<int>>& triangle,
5                  vector<vector<int>>& memo) {
6     int n = triangle.size();
7     if (i == n - 1) return triangle[i][j];
8     if (memo[i][j] != -1) return memo[i][j];
9
10    int down      = triangleMemo(i + 1, j, triangle, memo);
11    int downRight = triangleMemo(i + 1, j + 1, triangle, memo);
12    return memo[i][j] = triangle[i][j] + min(down, downRight);
13 }
14
15 int main() {
16     vector<vector<int>> triangle = {{2},{3,4},{6,5,7},{4,1,8,3}};
17     int n = triangle.size();
18     vector<vector<int>> memo(n, vector<int>(n, -1));
19     cout << triangleMemo(0, 0, triangle, memo) << endl; // 11
20     return 0;
21 }

```

### Approach 2 Complexity

**Time.**  $O(n^2)$  – the triangle has  $O(n^2)$  total cells. **Space.**  $O(n^2)$  memo +  $O(n)$  recursion stack.

### Approach 3: Tabulation

#### Fill the Table Bottom-Up, in the Direction of the Base Case

Because the recurrence reads from row  $i + 1$  to compute row  $i$ , the tabulation loop must process rows in **decreasing** order of  $i$ , starting by copying the last row directly (the base case), and working upward to row 0 – the mirror image of Sections 1.2–1.4’s top-to-bottom fill order.

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int triangleTabulation(vector<vector<int>>& triangle) {
5     int n = triangle.size();
6     vector<vector<int>> dp(n, vector<int>(n, 0));
7
8     for (int j = 0; j < n; j++) dp[n-1][j] = triangle[n-1][j];
9
10    for (int i = n - 2; i >= 0; i--) {
11        for (int j = 0; j <= i; j++) {
12            int down = dp[i+1][j];
13            int downRight = dp[i+1][j+1];
14            dp[i][j] = triangle[i][j] + min(down, downRight);
15        }
16    }
17    return dp[0][0];
18 }
19
20 int main() {
21     vector<vector<int>> triangle = {{2},{3,4},{6,5,7},{4,1,8,3}};
22     cout << triangleTabulation(triangle) << endl; // 11
23     return 0;
24 }

```

### Dry run of the tabulation table

**Dry Run.** Take  $triangle = [[2], [3, 4], [6, 5, 7], [4, 1, 8, 3]]$ .

- Row 3 (base case):  $dp[3] = [4, 1, 8, 3]$ .
- Row 2:  $dp[2][0] = 6 + \min(4, 1) = 7$ ;  $dp[2][1] = 5 + \min(1, 8) = 6$ ;  $dp[2][2] = 7 + \min(8, 3) = 10$ .
- Row 1:  $dp[1][0] = 3 + \min(7, 6) = 9$ ;  $dp[1][1] = 4 + \min(6, 10) = 10$ .
- Row 0:  $dp[0][0] = 2 + \min(9, 10) = 11$ .

The minimum path sum is 11, via  $2 \rightarrow 3 \rightarrow 5 \rightarrow 1$ .

### Approach 3 Complexity

**Time.**  $O(n^2)$ . **Space.**  $O(n^2)$  table, no recursion stack.

### Approach 4: Space Optimization

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int triangleSpaceOptimized(vector<vector<int>>& triangle) {
5     int n = triangle.size();
6     vector<int> nextRow = triangle[n - 1];
7
8     for (int i = n - 2; i >= 0; i--) {
9         vector<int> currentRow(i + 1);
10        for (int j = 0; j <= i; j++) {
11            int down = nextRow[j];

```

```

12         int downRight = nextRow[j + 1];
13         currentRow[j] = triangle[i][j] + min(down, downRight);
14     }
15     nextRow = currentRow;
16 }
17 return nextRow[0];
18 }
19
20 int main() {
21     vector<vector<int>> triangle = {{2},{3,4},{6,5,7},{4,1,8,3}};
22     cout << triangleSpaceOptimized(triangle) << endl; // 11
23     return 0;
24 }

```

### Approach 4 Complexity

**Time.**  $O(n^2)$ . **Space.**  $O(n)$  for the single rolling row.

### Key Takeaway

Always let the *base case's location* dictate the tabulation's fill direction – when a recurrence looks “forward” to a later row (as here), the table must be filled starting from that later base case and working backward, not from index 0 forward. Checking which direction the recursion's base case sits in, before writing the tabulation loop, is a habit worth forming explicitly to avoid reading uninitialized table entries.

## 1.6 Ninja's Training

**Problem Statement.** A ninja has  $n$  days of training. On each day, they must perform exactly one of 3 activities (indexed 0, 1, 2), earning  $points[day][activity]$  points – but they may **not** perform the **same** activity on two consecutive days. Maximize the total points earned over all  $n$  days.

### Pattern Recognition

“Choose one of several options each day, but not the same option as the previous day” is the keyword pattern from Section 1.1 for a DP state with a **non-spatial second dimension**: the state needs both *which day* we are on and *which activity was excluded* (because it was used the day before), even though there is no grid or spatial structure here at all – the “two dimensions” are day and constraint, not row and column.

### Deriving the recurrence

Let  $f(day, last)$  be the maximum points earnable from day 0 through day  $day$ , given that activity  $last$  is forbidden on day  $day$  specifically (because it was used on day  $day + 1$ , which we are building backward from – or, viewed forward, because it was the most recently committed activity choice we must avoid repeating). For each candidate activity  $task \neq last$ :

$$f(day, last) = \max_{task \neq last} \left( points[day][task] + f(day - 1, task) \right).$$

Base case:  $f(0, last) = \max_{task \neq last} points[0][task]$  (no earlier day to recurse into). The final answer is  $f(n - 1, 3)$ , using a sentinel value 3 (outside the valid activity range  $\{0, 1, 2\}$ ) to mean “no activity is forbidden on the last day,” since nothing comes after it.

## Approach 1: Recursion

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int ninjaTrainingRecursive(int day, int last, vector<vector<int>>&
   points) {
5     if (day == 0) {
6         int best = 0;
7         for (int task = 0; task < 3; task++) {
8             if (task != last) best = max(best, points[0][task]);
9         }
10        return best;
11    }
12
13    int best = 0;
14    for (int task = 0; task < 3; task++) {
15        if (task != last) {
16            int candidate = points[day][task] +
17                ninjaTrainingRecursive(day - 1, task, points);
18            best = max(best, candidate);
19        }
20    }
21    return best;
22 }
23
24 int main() {
25     vector<vector<int>> points = {{10,40,70},{20,50,80},{30,60,90}};
26     int n = points.size();
27     cout << ninjaTrainingRecursive(n - 1, 3, points) << endl; // 210
28     return 0;
29 }

```

### Approach 1 Complexity

**Time.**  $O(3^n)$  – up to 3 branches per call, depth  $n$ . **Space.**  $O(n)$  recursion stack.

## Approach 2: Memoization

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int ninjaTrainingMemo(int day, int last, vector<vector<int>>& points,
   vector<vector<int>>& memo) {
5
6     if (day == 0) {
7         int best = 0;
8         for (int task = 0; task < 3; task++) {
9             if (task != last) best = max(best, points[0][task]);
10        }
11        return best;
12    }
13    if (memo[day][last] != -1) return memo[day][last];
14
15    int best = 0;
16    for (int task = 0; task < 3; task++) {
17        if (task != last) {

```

```

18         int candidate = points[day][task] +
19             ninjaTrainingMemo(day - 1, task, points, memo);
20         best = max(best, candidate);
21     }
22 }
23 return memo[day][last] = best;
24 }
25
26 int main() {
27     vector<vector<int>> points = {{10,40,70},{20,50,80},{30,60,90}};
28     int n = points.size();
29     vector<vector<int>> memo(n, vector<int>(4, -1));
30     cout << ninjaTrainingMemo(n - 1, 3, points, memo) << endl; // 210
31     return 0;
32 }

```

### Approach 2 Complexity

**Time.**  $O(n \cdot 4 \cdot 3) = O(n)$  –  $n$  days, 4 possible *last* values, 3 choices of *task* per state.  
**Space.**  $O(n)$  memo (the 4 factor is a constant) +  $O(n)$  recursion stack.

### Approach 3: Tabulation

```

1  #include <bits/stdc++.h>
2  using namespace std;
3
4  int ninjaTrainingTabulation(vector<vector<int>>& points) {
5      int n = points.size();
6      vector<vector<int>> dp(n, vector<int>(4, 0));
7
8      for (int last = 0; last < 4; last++) {
9          int best = 0;
10         for (int task = 0; task < 3; task++) {
11             if (task != last) best = max(best, points[0][task]);
12         }
13         dp[0][last] = best;
14     }
15
16     for (int day = 1; day < n; day++) {
17         for (int last = 0; last < 4; last++) {
18             int best = 0;
19             for (int task = 0; task < 3; task++) {
20                 if (task != last) {
21                     int candidate = points[day][task] + dp[day-1][task];
22                     best = max(best, candidate);
23                 }
24             }
25             dp[day][last] = best;
26         }
27     }
28     return dp[n - 1][3];
29 }
30
31 int main() {
32     vector<vector<int>> points = {{10,40,70},{20,50,80},{30,60,90}};

```

```

33     cout << ninjaTrainingTabulation(points) << endl; // 210
34     return 0;
35 }

```

## Dry run of the tabulation table

**Dry Run.** Take  $points = [[10, 40, 70], [20, 50, 80], [30, 60, 90]]$ .

| day | last = 0 | last = 1 | last = 2 | last = 3 |
|-----|----------|----------|----------|----------|
| 0   | 70       | 70       | 40       | 70       |
| 1   | 120      | 120      | 120      | 120      |
| 2   | 210      | 210      | 180      | 210      |

Reading  $dp[0][2] = 40$ : with activity 2 forbidden on day 0, the best of  $\{points[0][0], points[0][1]\} = \{10, 40\}$  is 40. Reading  $dp[1][3]$  (no restriction on day 1):  $\max(20 + dp[0][0], 50 + dp[0][1], 80 + dp[0][2]) = \max(20+70, 50+70, 80+40) = \max(90, 120, 120) = 120$ . Reading the final answer,  $dp[2][3]$ :  $\max(30 + dp[1][0], 60 + dp[1][1], 90 + dp[1][2]) = \max(30+120, 60+120, 90+120) = \max(150, 180, 210) = 210$ .

### Approach 3 Complexity

**Time.**  $O(n)$  (the  $4 \times 3$  factor is a constant). **Space.**  $O(n)$  table, no recursion stack.

## Approach 4: Space Optimization

```

1  #include <bits/stdc++.h>
2  using namespace std;
3
4  int ninjaTrainingSpaceOptimized(vector<vector<int>>& points) {
5      int n = points.size();
6      vector<int> prevDay(4, 0);
7
8      for (int last = 0; last < 4; last++) {
9          int best = 0;
10         for (int task = 0; task < 3; task++) {
11             if (task != last) best = max(best, points[0][task]);
12         }
13         prevDay[last] = best;
14     }
15
16     for (int day = 1; day < n; day++) {
17         vector<int> currentDay(4, 0);
18         for (int last = 0; last < 4; last++) {
19             int best = 0;
20             for (int task = 0; task < 3; task++) {
21                 if (task != last) {
22                     best = max(best, points[day][task] + prevDay[task]);
23                 }
24             }
25             currentDay[last] = best;
26         }
27         prevDay = currentDay;
28     }
29     return prevDay[3];

```

```

30 }
31
32 int main() {
33     vector<vector<int>> points = {{10,40,70},{20,50,80},{30,60,90}};
34     cout << ninjaTrainingSpaceOptimized(points) << endl; // 210
35     return 0;
36 }

```

#### Approach 4 Complexity

**Time.**  $O(n)$ . **Space.**  $O(1)$  (a fixed-size array of 4 entries, independent of  $n$ ).

#### Key Takeaway

A DP state's second dimension does not need to be spatial – here, it is “which choice is currently forbidden,” a small, fixed-size constraint dimension exactly analogous to the visited-tracking state used in backtracking, but folded directly into the DP table's indexing instead of an external visited structure. Whenever a problem's choice at step  $i$  is constrained only by the choice at step  $i - 1$  (not by anything further back), this “day  $\times$  last choice” state shape applies directly.

## 1.7 Ninja and his Friends (3D DP / Cherry Pickup)

**Problem Statement.** Given an  $m \times n$  grid of chocolates, two collectors start at the top row – one at  $(0, 0)$ , the other at  $(0, n - 1)$  – and both move **simultaneously**, one row at a time, down to row  $m - 1$ . From  $(row, col)$ , a collector may move to  $(row + 1, col - 1)$ ,  $(row + 1, col)$ , or  $(row + 1, col + 1)$ . Each collects the chocolates in every cell it visits; if both collectors occupy the **same** cell at the same time, that cell's chocolates are counted only **once**. Maximize the total chocolates collected.

#### Pattern Recognition

This closing problem of the chapter is a deliberate capstone: “*two agents moving simultaneously through the same grid*” is Section 1.1's 3D DP keyword pattern. The state must track **three** coordinates – the shared row (since both move in lockstep, one row per step) and *each* collector's column – because the optimal move for one collector can depend on where the other currently is (specifically, whether they currently coincide, which determines whether a cell's chocolates are double-counted).

### Deriving the recurrence

Let  $f(row, col1, col2)$  be the maximum chocolates collectible from row  $row$  down to the last row, given that collector 1 is currently at column  $col1$  and collector 2 is currently at column  $col2$  (both in the same row, since they move together). The chocolates gained at this row are  $grid[row][col1]$  plus  $grid[row][col2]$  *unless*  $col1 = col2$ , in which case the cell's value is added only once. Each collector independently chooses one of 3 column moves  $(-1, 0, +1)$  for the next row, giving  $3 \times 3 = 9$  combinations to maximize over:

$$f(row, col1, col2) = collected + \max_{\substack{d_1, d_2 \in \{-1, 0, 1\} \\ \text{both columns valid}}} f(row + 1, col1 + d_1, col2 + d_2),$$

where  $collected = grid[row][col1] + (col1 \neq col2 ? grid[row][col2] : 0)$ . Base case: at the last row ( $row = m - 1$ ), there is no further move, so  $f(m - 1, col1, col2) = collected$  directly. The overall answer is  $f(0, 0, n - 1)$ .

## Approach 1: Recursion

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int solve(int row, int col1, int col2, vector<vector<int>>& grid, int m
, int n) {
5     if (col1 < 0 || col1 >= n || col2 < 0 || col2 >= n) return INT_MIN;
6
7     int collected = grid[row][col1];
8     if (col1 != col2) collected += grid[row][col2];
9
10    if (row == m - 1) return collected;
11
12    int best = INT_MIN;
13    for (int d1 = -1; d1 <= 1; d1++) {
14        for (int d2 = -1; d2 <= 1; d2++) {
15            int next = solve(row + 1, col1 + d1, col2 + d2, grid, m, n)
;
16            if (next != INT_MIN) best = max(best, next);
17        }
18    }
19    return collected + best;
20 }
21
22 int main() {
23     vector<vector<int>> grid = {{3,1,1},{2,5,1},{1,5,5},{2,1,1}};
24     int m = grid.size(), n = grid[0].size();
25     cout << solve(0, 0, n - 1, grid, m, n) << endl; // 24
26     return 0;
27 }

```

### Approach 1 Complexity

**Time.**  $O(9^m)$  – up to 9 branches per call, depth  $m$ . **Space.**  $O(m)$  recursion stack.

## Approach 2: Memoization

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int solveMemo(int row, int col1, int col2, vector<vector<int>>& grid,
5             int m, int n, vector<vector<vector<int>>>& memo) {
6     if (col1 < 0 || col1 >= n || col2 < 0 || col2 >= n) return INT_MIN;
7
8     int collected = grid[row][col1];
9     if (col1 != col2) collected += grid[row][col2];
10
11    if (row == m - 1) return collected;
12    if (memo[row][col1][col2] != -1) return memo[row][col1][col2];
13
14    int best = INT_MIN;
15    for (int d1 = -1; d1 <= 1; d1++) {
16        for (int d2 = -1; d2 <= 1; d2++) {
17            int next = solveMemo(row + 1, col1 + d1, col2 + d2, grid, m
, n, memo);

```

```

18         if (next != INT_MIN) best = max(best, next);
19     }
20 }
21 return memo[row][col1][col2] = collected + best;
22 }
23
24 int main() {
25     vector<vector<int>> grid = {{3,1,1},{2,5,1},{1,5,5},{2,1,1}};
26     int m = grid.size(), n = grid[0].size();
27     vector<vector<vector<int>>> memo(m, vector<vector<int>>(n, vector<
28         int>(n, -1)));
29     cout << solveMemo(0, 0, n - 1, grid, m, n, memo) << endl; // 24
30     return 0;
31 }

```

### Approach 2 Complexity

**Time.**  $O(m \cdot n^2 \cdot 9) = O(m \cdot n^2) - m \cdot n^2$  distinct states, each doing  $O(9) = O(1)$  work.

**Space.**  $O(m \cdot n^2)$  memo +  $O(m)$  recursion stack.

### Approach 3: Tabulation

```

1  #include <bits/stdc++.h>
2  using namespace std;
3
4  int solveTabulation(vector<vector<int>>& grid) {
5      int m = grid.size(), n = grid[0].size();
6      vector<vector<vector<int>>> dp(m, vector<vector<int>>(n, vector<int>
7          >(n, 0)));
8
9      for (int col1 = 0; col1 < n; col1++) {
10         for (int col2 = 0; col2 < n; col2++) {
11             dp[m-1][col1][col2] = grid[m-1][col1] +
12                 (col1 != col2 ? grid[m-1][col2] : 0);
13         }
14     }
15
16     for (int row = m - 2; row >= 0; row--) {
17         for (int col1 = 0; col1 < n; col1++) {
18             for (int col2 = 0; col2 < n; col2++) {
19                 int collected = grid[row][col1] +
20                     (col1 != col2 ? grid[row][col2] : 0);
21
22                 int best = INT_MIN;
23                 for (int d1 = -1; d1 <= 1; d1++) {
24                     for (int d2 = -1; d2 <= 1; d2++) {
25                         int nc1 = col1 + d1, nc2 = col2 + d2;
26                         if (nc1 >= 0 && nc1 < n && nc2 >= 0 && nc2 < n)
27                             {
28                                 best = max(best, dp[row+1][nc1][nc2]);
29                             }
30                     }
31                 }
32                 dp[row][col1][col2] = collected + (best == INT_MIN ? 0
33                     : best);
34             }
35         }
36     }
37 }

```

```

32     }
33 }
34 return dp[0][0][n-1];
35 }
36
37 int main() {
38     vector<vector<int>> grid = {{3,1,1},{2,5,1},{1,5,5},{2,1,1}};
39     cout << solveTabulation(grid) << endl; // 24
40     return 0;
41 }

```

## Dry run

**Dry Run.** Take  $grid = \begin{bmatrix} 3 & 1 & 1 \\ 2 & 5 & 1 \\ 1 & 5 & 5 \\ 2 & 1 & 1 \end{bmatrix}$ .

The last row's table is filled directly: e.g.  $dp[3][0][2] = grid[3][0] + grid[3][2] = 2 + 1 = 3$  (collectors at different columns, both counted). Working upward,  $dp[2][.][.]$  combines row 2's chocolates with the best reachable entry of  $dp[3]$ , and so on up to  $dp[0][0][2]$ . Tracing the full optimal path (confirmed by the code): collector 1 follows  $0 \rightarrow 1 \rightarrow 2 \rightarrow 1$  (columns), collector 2 follows  $2 \rightarrow 1 \rightarrow 2 \rightarrow 1$ , jointly visiting cells that sum to a maximum of 24 chocolates.

### Approach 3 Complexity

**Time.**  $O(m \cdot n^2)$ . **Space.**  $O(m \cdot n^2)$  table, no recursion stack.

## Approach 4: Space Optimization

### Only the Next Row's 2D Slice Is Ever Needed

Exactly as in Section 1.2, row  $row$ 's table only ever reads from row  $row + 1$  – so the full 3D table collapses to two rolling 2D slices, each of size  $n \times n$ , instead of  $m \times n \times n$ .

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int solveSpaceOptimized(vector<vector<int>>& grid) {
5     int m = grid.size(), n = grid[0].size();
6     vector<vector<int>> nextRow(n, vector<int>(n, 0));
7
8     for (int col1 = 0; col1 < n; col1++)
9         for (int col2 = 0; col2 < n; col2++)
10             nextRow[col1][col2] = grid[m-1][col1] +
11                 (col1 != col2 ? grid[m-1][col2] : 0);
12
13     for (int row = m - 2; row >= 0; row--) {
14         vector<vector<int>> currentRow(n, vector<int>(n, 0));
15         for (int col1 = 0; col1 < n; col1++) {
16             for (int col2 = 0; col2 < n; col2++) {
17                 int collected = grid[row][col1] +
18                     (col1 != col2 ? grid[row][col2] : 0);
19
20                 int best = INT_MIN;
21                 for (int d1 = -1; d1 <= 1; d1++) {

```

```

22         for (int d2 = -1; d2 <= 1; d2++) {
23             int nc1 = col1 + d1, nc2 = col2 + d2;
24             if (nc1 >= 0 && nc1 < n && nc2 >= 0 && nc2 < n)
25                 {
26                     best = max(best, nextRow[nc1][nc2]);
27                 }
28         }
29         currentRow[col1][col2] = collected + (best == INT_MIN ?
30             0 : best);
31     }
32     nextRow = currentRow;
33 }
34 return nextRow[0][n-1];
35 }
36
37 int main() {
38     vector<vector<int>> grid = {{3,1,1},{2,5,1},{1,5,5},{2,1,1}};
39     cout << solveSpaceOptimized(grid) << endl; // 24
40     return 0;
41 }

```

#### Approach 4 Complexity

**Time.**  $O(m \cdot n^2)$ . **Space.**  $O(n^2)$  for two rolling 2D slices, instead of  $O(m \cdot n^2)$  for the full 3D table.

#### Key Takeaway

A 3D DP state is exactly what “ $k$  agents moving through the same structure simultaneously” demands: one shared dimension for the synchronized step (here, the row both collectors are always on together), plus one dimension per agent for its own position. Everything about the four-stage methodology – recursion first, then memoization, then tabulation in the base case’s direction, then collapsing to rolling slices – is unchanged from the simpler 1D and 2D problems earlier in this book; only the number of dimensions being tracked grows.